Customer Risk Segmentation via Cluster Analysis

K-means and DBSCAN on a Simulated Retail Banking Dataset

Your Name

github.com/your-username/customer-risk-segmentation

Dataset: 100,300 synthetic customer records (100,000 unique customers), fixed random seed 602 for reproducibility

Personal data science portfolio project

1 Introduction and Segmentation Question

Modern retail banks routinely analyse customer-level financial data to support strategic decisions, including marketing, retention, and risk management. This report applies unsupervised cluster analysis to a simulated retail banking dataset of 100,000 customers to address the following segmentation question:

> **"Can customers be grouped into distinct, actionable risk profiles based on their income, savings, debt, and repayment behaviour, in order to support differentiated credit risk management and collections strategy?"**

The dataset was generated using a fixed random seed (602) via a supplied R simulation function, producing demographic, product-holding, transactional, credit-risk, and engagement variables, together with realistic data-quality issues (missing values, entry errors, and duplicate records) that were identified and corrected as part of this analysis (Section 3).

Two complementary clustering algorithms were applied: K-means, to establish a small number of broad, business-interpretable risk segments; and DBSCAN, a density-based method used to identify anomalous, high-risk customers that a centroid-based method would otherwise absorb into an average cluster. The remainder of this report documents variable selection, data preparation, methodology, results, cluster interpretation, business recommendations, a comparison of the two methods, and limitations.

2 Variable Selection and Justification

2.1 Selected Variables

Nine variables were selected for the cluster analysis, chosen to jointly represent three complementary dimensions of financial risk:

- **Repayment capacity**: `monthly_income`, `savings_balance`, `current_balance` — buffers against financial shocks.
- **Leverage**: `total_outstanding_debt`, `debt_to_income_ratio`, `num_active_loans` — current indebtedness relative to means.
- **Repayment behaviour**: `num_missed_payments`, `overdraft_usage_yearly`, `credit_score` — realised repayment conduct.

Combining capacity, leverage, and behaviour avoids a one-dimensional view of risk and allows segments to separate, for example, "high income but high leverage" customers from "low income but low leverage, reliable payers" — a distinction a single variable could not draw.

2.2 Excluded Variables

Demographic fields (age, gender, region, education, marital status) were excluded as they are not direct financial-risk indicators and their use in risk segmentation raises proxy-discrimination concerns. Identifier fields (`customer_id`, `branch_id`, `relationship_manager_id`) were dropped as meaningless for clustering. Engagement and service variables (`satisfaction_score`, `branch_visits_yearly`, `num_complaints`, `churned`, `has_mobile_banking`) were excluded as they reflect service experience rather than credit risk. `loan_default_flag` was excluded because it is the eventual supervised-learning target; including it would leak the outcome into an unsupervised risk-clustering exercise.

2.3 Variables Rejected for Correlation / Redundancy

`has_personal_loan` and `has_mortgage` were dropped because they are already summarised in `num_active_loans`. `avg_transaction_value` and `avg_monthly_transactions` were excluded as they correlate strongly with `monthly_income` and mainly reflect channel activity rather than risk.

3 Data Cleaning and Preprocessing

3.1 Identified Data Quality Issues

Three distinct issues were identified by direct inspection of the nine selected variables in the generated dataset.

Issue 1 — Exact duplicate records

300 fully duplicated rows (0.3% of 100,300 records) were found, consistent with a data-integration duplication error. **Remedy:** removed via `distinct()` row de-duplication. This was preferred over fuzzy matching because the rows were byte-for-byte identical, so exact removal loses no information and introduces no bias.

Issue 2 — Implausible / impossible values (entry errors)

- `monthly_income`: 176 values were non-positive (including −500) or implausibly extreme (up to 5,000,000, against a clean 99.9th percentile of ∼29,068). Recoded to `NA` and handled with Issue 3.
- `debt_to_income_ratio`: 25 negative values (a ratio cannot be negative). Corrected via `abs()`, which exactly reverses a sign-flip entry error.
- `savings_balance`: ∼500 values inflated roughly 1000×, consistent with an extra-zero entry error. Winsorised (capped) at the 99.5th percentile rather than deleted, preserving the customer record while preventing a handful of rows from dominating distance-based clustering.

Capping/correction was preferred over deletion in each case because the affected customers likely have genuine, otherwise-usable records; deleting them would needlessly shrink the sample and could bias segments if the errors are not random.

Issue 3 — Missing values

`monthly_income` was missing in 6.0% of records and `credit_score` in 4.0%. **Remedy:** median imputation, with a companion `*_was_missing` flag retained for each variable. Median imputation was preferred over mean imputation because both variables are heavily right-skewed, so the mean is pulled upward by extreme earners while the median better represents a typical customer. Listwise deletion was rejected as it would discard roughly 10% of customers combined and could bias segments if missingness is related to customer type (e.g. unverified self-employed income).

3.2 Preprocessing: Transformation and Scaling

Skewness and spread were checked empirically for each variable before choosing a scaling method, rather than applying one method uniformly.

Variable group	Variables	Skew (raw → log)	Method chosen
Monetary (right-skewed)	income, savings, current balance, debt, DTI	up to 25.6 → −0.6 to 4.2	log1p, then Z-score
Bounded, symmetric spread	credit_score	low	Robust scaling (median/IQR)
Zero-inflated counts	active loans, missed payments, overdraft use	median = IQR = 0	Min-Max normalisation

Table 1: Preprocessing method chosen per variable group.

The five monetary variables were log1p-transformed to compress heavy right tails (a handful of wealthy customers would otherwise dominate Euclidean-distance clustering), then Z-scored so all five carry comparable variance. `credit_score` has a genuinely symmetric spread with non-zero IQR, so median/IQR robust scaling was used directly. The three count variables (`num_active_loans`,

`num_missed_payments`, `overdraft_usage_yearly`) are heavily zero-inflated — checking directly confirmed their median *and* IQR are both 0 ($\geq$75% of customers have zero missed payments or overdraft use), which makes robust scaling mathematically degenerate; Min-Max normalisation was used instead, as it bounds each variable to $[0, 1]$ without relying on a spread statistic. `debt_to_income_ratio` and `total_outstanding_debt` retain moderate residual skew even after logging, reflecting a structural point mass at zero (customers with no debt) rather than a preprocessing failure — noted as a limitation in Section 8.

4 Methodology

4.1 Clustering Algorithms Employed

Two complementary algorithms were applied to the preprocessed 9-feature, 100,000-row matrix:

- **K-means** — a centroid-based partitioning method, used to define a small number of broad, interpretable customer segments for the risk-classification business objective.
- **DBSCAN** — a density-based method, used as the second algorithm to detect anomalous, high-risk customers as explicit "noise" rather than forcing them into an averaged cluster.

4.2 Choice of Parameters

K-means was run with `nstart = 10` (10 random centroid initialisations per k, keeping the best solution) and a maximum of 50 iterations, for $k = 1$ to 10, using `set.seed(602)` for reproducibility.

For DBSCAN, `minPts` was set to 18 (twice the feature dimensionality, a standard rule of thumb for continuous, moderate-dimensional data). `eps = 0.60` was chosen from the k-distance elbow: the sorted 18th-nearest-neighbour distance curve rises gently up to roughly the 90th percentile (0.568) before accelerating sharply beyond the 96th percentile (0.725+); 0.60 sits just past the knee, balancing a meaningful density threshold against an excessive noise fraction.

4.3 Elbow and Silhouette Analyses

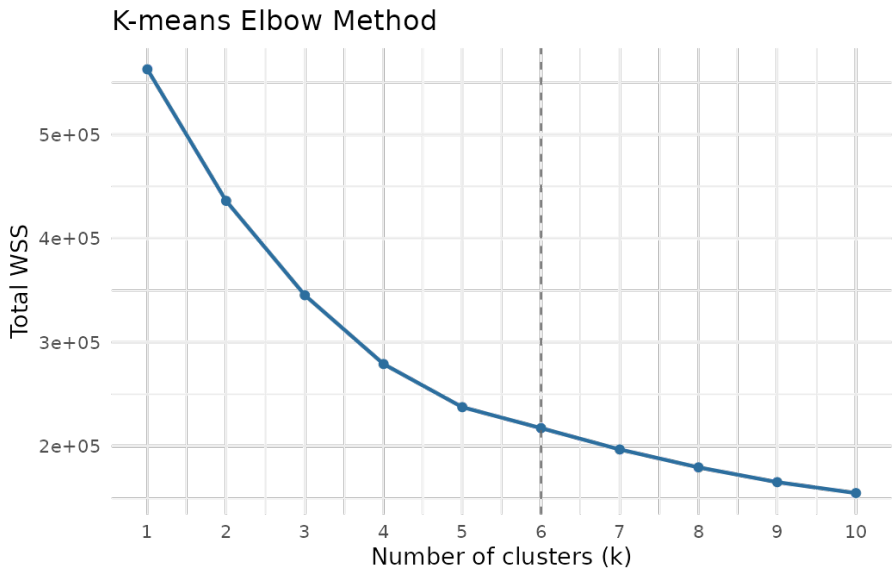

Figure 1: K-means elbow method — total within-cluster sum of squares (WSS) by k.

Total WSS declines steeply from $k = 2$ to $k = 5$ (19–22% per additional cluster) and flattens markedly from $k = 6$ onward (7–9% per step) — the elbow sits at $k = 5$ or 6.

The global maximum average silhouette width occurs at $k = 2$ (0.293), but a clear local peak appears at $k = 6$–7 (0.286–0.288), nearly as high, following a dip at $k = 3$.

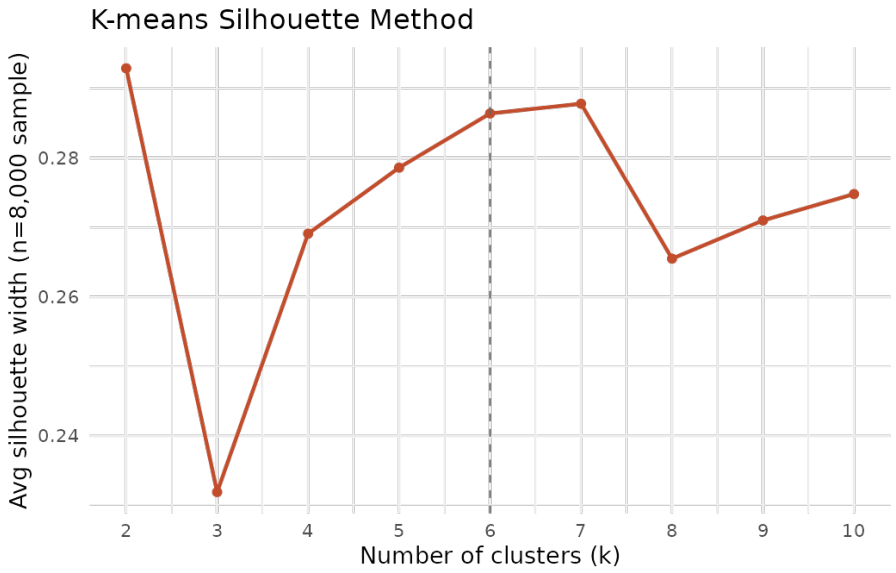

Figure 2: K-means silhouette method — average silhouette width by k (evaluated on an 8,000-customer sample; silhouette computation is $O(n^2)$ and infeasible on the full 100,000 rows).

Reconciling the two diagnostics

The two methods nominally disagree (elbow ≈ 5–6; silhouette's global optimum = 2), so both were checked against the business objective before a final choice was made. $k = 2$ is statistically the tightest split, but profiling shows it merely separates "low outstanding debt" (71% of customers) from "high debt / elevated DTI" (29%) — too coarse to support differentiated risk strategy. $k = 6$ sits at a local silhouette peak nearly matching the global maximum, coincides with where the elbow's marginal gains flatten, and — critically — produces six segments that are individually interpretable and include a distinct, small, high-risk cluster (Section 6). $k = 6$ was therefore selected as the final solution, prioritising business actionability where the two purely statistical diagnostics were closely matched.

5 Results

5.1 Cluster Visualisations

No dendrogram is included, as hierarchical clustering was not one of the two algorithms used for this analysis (K-means and DBSCAN were chosen instead, for the reasons discussed in Section 4.1).

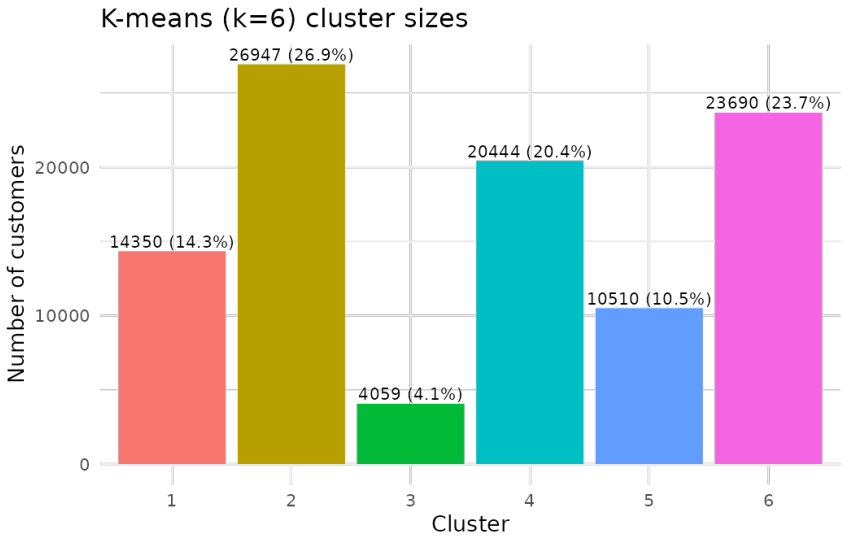

Figure 3: K-means ($k = 6$) cluster sizes.

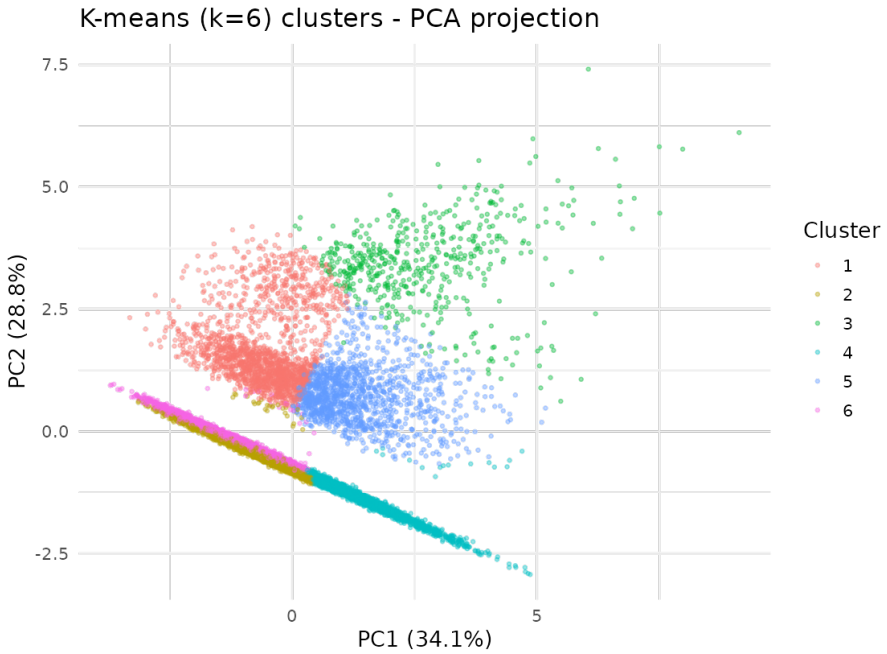

Figure 4: K-means ($k = 6$) clusters projected onto the first two principal components (PC1 = 34.1%, PC2 = 28.8% of variance; 12,000-customer plotting sample).

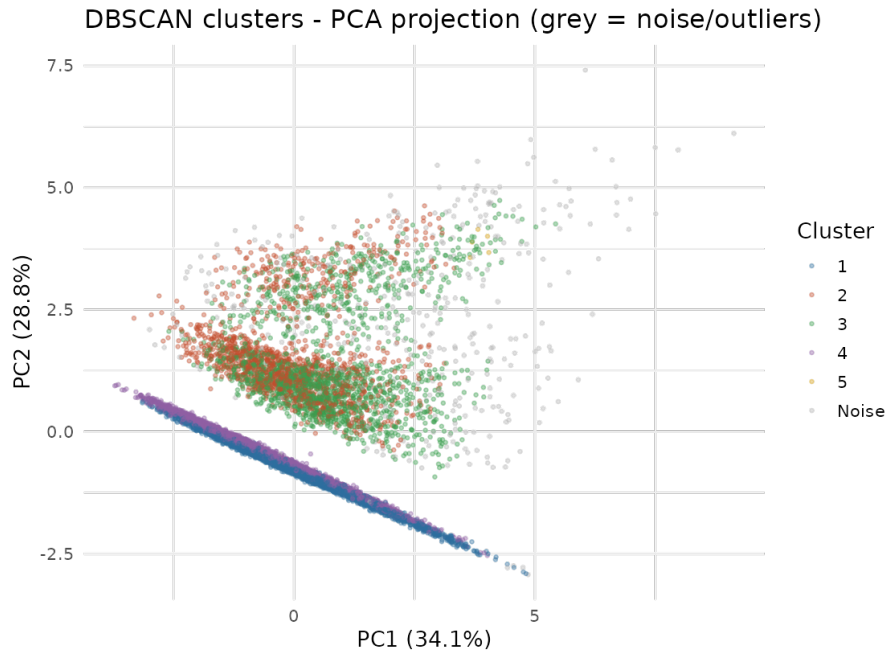

Figure 5: DBSCAN clusters projected onto the same PCA axes; grey points are density-based outliers ("noise").

5.2 Summary Tables

K-means ($k = 6$) cluster profile (raw-scale means)

Cl.	n (%)	Income	Savings	Debt	DTI	Credit	Missed	OD/yr
1	14350 (14.3%)	$6942.83	$56877.49	$38713.60	0.47	701.51	0.06	0.56
2	26947 (26.9%)	$5458.54	$36607.79	$2.85	0.00	707.18	0.01	0.01
3	4059 (4.1%)	$1980.51	$11390.35	$147483.04	11.57	661.09	2.27	0.21
4	20444 (20.4%)	$1108.34	$3119.28	$3.27	0.00	647.88	0.13	0.01
5	10510 (10.5%)	$2080.83	$6853.64	$8984.40	0.48	641.95	0.24	0.94
6	23690 (23.7%)	$5080.07	$40889.19	$3.60	0.00	702.21	0.02	0.01

Table 2: K-means cluster profile on raw (unscaled) variables. OD/yr = overdraft usage per year.

DBSCAN: noise (outlier) vs. clustered customers (raw-scale means)

Variable	Noise (n=3,303)	Clustered (n=96,697)
monthly_income	2766.63	4245.19
savings_balance	43539.90	29055.20
current_balance	2407.31	2359.76
total_outstanding_debt	80499.26	10165.15
debt_to_income_ratio	9.28	0.29
credit_score	633.06	686.09
num_active_loans	1.08	0.34
num_missed_payments	2.03	0.10
overdraft_usage_yearly	0.90	0.17

Table 3: DBSCAN noise points vs. clustered customers on raw risk indicators.

6 Cluster Profiles and Business Labels

Cluster 2 — "High-Income Savers, No Debt" (26.9%, n=26,947)

Highest credit score (707), zero debt, no current account, income $5,459. The largest, lowest-risk segment: financially comfortable customers who save rather than borrow. **Business label: Minimal risk**

Cluster 6 — "Affluent Transactors, No Debt" (23.7%, n=23,690)

Similar to Cluster 2 but distinguished by active current-account usage (highest current balance, $6,594) alongside zero debt and strong credit (702). Engaged, debt-free, day-to-day banking relationship. **Business label: Minimal risk**

Cluster 4 — "Underbanked, Low-Income" (20.4%, n=20,444)

Lowest income ($1,108) and lowest savings ($3,119), essentially no debt, but the weakest credit score (648) of any segment. Low risk today mainly because these customers carry no debt, not because of strong credit fundamentals. **Business label: Low risk, but fragile**

Cluster 1 — "Affluent Leveraged (Mortgage-Holders)" (14.3%, n=14,350)

High income ($6,943) and savings ($56,877) alongside meaningful debt ($38,714, DTI 0.47) — consistent with mortgage or large personal-loan holders. Good credit (702), very low missed payments (0.06). **Business label: Low-moderate risk**

Cluster 5 — "Overdraft-Reliant, Moderate Risk" (10.5%, n=10,510)

Modest income ($2,081), one of the lowest credit scores (642), and by far the highest overdraft usage (0.94/year). Debt is moderate (DTI 0.48) but repeated overdraft reliance signals cash-flow stress. **Business label: Moderate risk**

Cluster 3 — "Financially Distressed" (4.1%, n=4,059)

The clear high-risk segment: low income ($1,981) combined with very high debt ($147,483) and an extreme debt-to-income ratio (11.57 — roughly 25× the next-highest cluster). Highest missed-payment rate (2.27/year) of any segment. **Business label: High risk**

7 Interpretation, Recommendations, and Comparison of Methods

7.1 Interpretation and Business Recommendations

- Cluster 3 (Financially Distressed, 4.1%) should be prioritised for proactive risk management: early-warning monitoring, restructuring or hardship-programme outreach, and tighter approval thresholds on further lending given the extreme DTI and missed-payment rate.
- Cluster 5 (Overdraft-Reliant, 10.5%) is a moderate-risk, high-value intervention target: targeted cash-flow products (e.g. graduated overdraft limits, budgeting tools, or a consolidation loan) could reduce reliance on high-cost overdraft facilities before customers migrate toward Cluster 3.
- Clusters 2 and 6 (jointly 50.6%, debt-free, high credit) are prime candidates for cross-selling accretive products — credit cards, investment accounts, fixed deposits — since capacity is high and current risk exposure is minimal.
- Cluster 1 (Affluent Leveraged, 14.3%) represents the bank's mortgage/large-loan relationship base; retention and refinancing offers are more relevant here than risk mitigation, given strong credit and low missed payments despite high debt.

- Cluster 4 (Underbanked, 20.4%) is low-risk today but financially fragile (weak credit score, minimal buffers); appropriate for financial-inclusion products (starter credit-builder cards, low-fee savings) rather than collections attention.

7.2 Comparison of Clustering Methods

K-means and DBSCAN answered different but complementary parts of the segmentation question. K-means, by construction, assigns every customer to one of a fixed number of roughly spherical clusters, which produced six broad, evenly-sized, business-interpretable segments — well suited to designing differentiated marketing and relationship-management strategies for the bulk of the customer base.

DBSCAN made no assumption about cluster shape or count, and instead of forcing every customer into the nearest centroid, it explicitly separated 3.3% of customers (3,303 records) as density-based "noise". Comparing this noise set against the rest of the population on every raw risk variable (Table, Section 5.2) shows it is dramatically worse on every dimension simultaneously — 32× higher DTI, 8× more outstanding debt, and 20× more missed payments than the clustered majority. This is a materially higher concentration of risk than even K-means' own worst cluster (Cluster 3), because DBSCAN's noise definition specifically isolates points that are sparse in the full 9-dimensional feature space rather than merely distant from a single centroid.

In practice, the two methods are best used together: K-means for the primary segmentation used in day-to-day marketing and relationship strategy, and DBSCAN as a targeted overlay to flag the small number of customers who most urgently warrant manual underwriting review or early intervention — a distinction K-means alone cannot make, since its centroid-based objective necessarily averages outliers into their nearest cluster rather than isolating them.

8 Limitations and Conclusion

8.1 Limitations

- **Simulated data**: relationships between variables (e.g. income → debt → default probability) were generated by parametric formulas rather than observed from real customer behaviour; cluster structure may not fully generalise to a live portfolio.
- **Residual skew**: `debt_to_income_ratio` and `total_outstanding_debt` retained moderate skew even after log-transformation, due to a structural point mass at zero (no-debt customers) that a single continuous transform cannot fully resolve; a two-part (zero-inflated) model was outside this project's scope.
- **Silhouette evaluated on a sample**: due to $O(n^2)$ computational cost, silhouette width was computed on an 8,000-customer sample rather than the full 100,000 rows; while cluster assignments themselves came from the full-data fit, the reported silhouette values carry sampling variability.
- **DBSCAN parameter sensitivity**: `eps` and `minPts` were chosen via a k-distance elbow heuristic and a standard rule of thumb, respectively; different choices materially affect the noise fraction and cluster count, and were not exhaustively grid-searched.
- **Static, cross-sectional segmentation**: clusters reflect a single snapshot of each customer's financial position and do not capture trajectory (e.g. a customer moving from Cluster 2 toward Cluster 5 over time), which would strengthen early-warning value.

8.2 Conclusion

This analysis identified six actionable K-means risk/value segments and a DBSCAN-based outlier overlay isolating the most acute risk cases, using nine variables spanning income, leverage, and repayment behaviour selected specifically for the risk-classification business objective. The two methods were shown to be complementary rather than redundant: K-means for portfolio-wide

segmentation strategy, and DBSCAN for surfacing the small minority of customers requiring urgent, individualised risk attention. Together they provide a practical foundation for differentiated marketing, retention, and collections strategy across the simulated customer base.

A Appendix — Fully Commented R Code

The complete, executable R pipeline (data generation, variable selection, cleaning, preprocessing, and cluster analysis) is reproduced below, extracted directly from the project's R Markdown source. Comment lines beginning with `#'` correspond to the narrative sections of the analysis; standard `#` comments mark individual processing steps.

```
#' ---
#' title: "BDAT602 Project -- Task 2: Customer Risk Segmentation via Cluster Analysis"
#' subtitle: "Data Generation, Variable Selection, Cleaning, Preprocessing, and Clustering"
#' author: "BDAT602 Group Project"
#' date: "`r format(Sys.Date(), '%d %B %Y')`"
#' output:
#'   html_document:
#'     toc: true
#'     toc_depth: 3
#'     toc_float: true
#'     number_sections: true
#'     theme: flatly
#' ---
#'
## ----setup, include=FALSE-----------------------------------------------------
knitr::opts_chunk$set(echo = TRUE, message = FALSE, warning = FALSE,
                      fig.width = 7.5, fig.height = 5, dpi = 150)
options(scipen = 999)

#'
#' # Data Generation
#'
#' The simulated retail banking dataset (~100,000 customers) is generated by the function below,
#' supplied for the project. It produces demographic, product-holding, transaction, credit-risk,
#' and engagement variables, together with a set of intentionally embedded data-quality issues
#' (missing values, outliers/entry errors, inconsistent labels, and duplicate rows) that are
#' identified and corrected in Section 3.
#'
## ----data-generation-function-------------------------------------------------
suppressMessages({
  library(dplyr)
  library(lubridate)
})

simulate_bdat602_project <- function(n = 100000, seed = 602) {
  set.seed(seed)

  # -- SECTION 1: IDENTIFIERS ------------------------------------------
  customer_id    <- 1:n
  branch_id      <- sample(1:300, n, replace = TRUE)
  relationship_manager_id <- sample(1:800, n, replace = TRUE)
  account_open_date <- as.Date("2024-12-31") -
    days(round(rlnorm(n, meanlog = 6.5, sdlog = 1.0)))

  # -- SECTION 2: DEMOGRAPHICS -----------------------------------------
  age <- pmax(18, pmin(85, round(rnorm(n, mean = 41, sd = 13))))

  gender <- sample(c("Male", "Female", "Other"), n, replace = TRUE,
                   prob = c(0.48, 0.50, 0.02))

  region <- sample(
    c("Greater Accra", "Ashanti", "Western", "Eastern",
      "Northern", "Volta", "Central", "Bono"),
    n, replace = TRUE,
    prob = c(0.30, 0.20, 0.12, 0.10, 0.10, 0.08, 0.06, 0.04)
  )

  education <- sample(
    c("None", "Primary", "Secondary", "Tertiary", "Postgraduate"),
    n, replace = TRUE,
    prob = c(0.04, 0.12, 0.32, 0.38, 0.14)
  )
```

```
63
64  employment_status <- sample(
65    c("Salaried", "Self-Employed", "Business Owner", "Student", "Unemployed", "Retired"),
66    n, replace = TRUE,
67    prob = c(0.42, 0.20, 0.13, 0.08, 0.07, 0.10)
68  )
69
70  marital_status <- sample(
71    c("Single", "Married", "Divorced", "Widowed"),
72    n, replace = TRUE,
73    prob = c(0.42, 0.45, 0.08, 0.05)
74  )
75
76  dependents <- pmax(0, round(rpois(n, lambda = ifelse(
77    marital_status == "Married", 2.0, 0.5
78  ))))
79
80  base_income <- case_when(
81    education == "None"          ~  600,
82    education == "Primary"       ~ 1000,
83    education == "Secondary"     ~ 2200,
84    education == "Tertiary"      ~ 4500,
85    education == "Postgraduate"  ~ 8000,
86    TRUE ~ 2200
87  )
88
89  monthly_income <- round(
90    base_income *
91      case_when(
92        employment_status == "Salaried"       ~ 1.00,
93        employment_status == "Self-Employed"  ~ 1.15,
94        employment_status == "Business Owner" ~ 1.45,
95        employment_status == "Student"        ~ 0.25,
96        employment_status == "Unemployed"     ~ 0.15,
97        employment_status == "Retired"        ~ 0.65,
98        TRUE ~ 1.00
99      ) *
100     ifelse(region %in% c("Greater Accra", "Ashanti"), 1.30, 1.00) *
101     rlnorm(n, meanlog = 0, sdlog = 0.40),
102   2
103 )
104
105 # -- SECTION 3: ACCOUNT & PRODUCT HOLDINGS ------------------------------
106 account_type <- sample(
107   c("Basic", "Standard", "Premium", "Private Banking"),
108   n, replace = TRUE,
109   prob = c(0.40, 0.35, 0.18, 0.07)
110 )
111
112 has_savings_account <- rbinom(n, 1, prob = 0.95)
113
114 has_current_account <- rbinom(n, 1, prob = plogis(
115   -1.0 + 0.9 * (account_type %in% c("Standard","Premium","Private Banking")) +
116     0.3 * (employment_status %in% c("Salaried","Business Owner"))
117 ))
118
119 has_fixed_deposit <- rbinom(n, 1, prob = plogis(
120   -2.0 + 0.7 * (account_type %in% c("Premium","Private Banking")) +
121     0.5 * (log1p(monthly_income) > 8)
122 ))
123
124 has_credit_card <- rbinom(n, 1, prob = plogis(
125   -2.2 + 1.0 * (account_type %in% c("Standard","Premium","Private Banking")) +
126     0.4 * (employment_status == "Salaried") +
127     0.3 * (age > 28)
128 ))
129
130 has_personal_loan <- rbinom(n, 1, prob = plogis(
131   -2.0 + 0.5 * (employment_status %in% c("Salaried","Business Owner")) +
132     0.3 * (dependents > 1) -
133     0.4 * (account_type == "Private Banking")
134 ))
135
136 has_mortgage <- rbinom(n, 1, prob = plogis(
137   -3.5 + 0.8 * (age > 30 & age < 60) +
```

```
      0.6 * (log1p(monthly_income) > 8.5) +
      0.5 * (marital_status == "Married")
  ))

  has_investment_account <- rbinom(n, 1, prob = plogis(
    -2.5 + 1.0 * (account_type %in% c("Premium","Private Banking")) +
      0.5 * (education %in% c("Tertiary","Postgraduate"))
  ))

  has_insurance_product <- rbinom(n, 1, prob = plogis(
    -1.8 + 0.5 * (has_mortgage == 1) +
      0.4 * (dependents > 0) +
      0.3 * (account_type != "Basic")
  ))

  has_mobile_banking <- rbinom(n, 1, prob = plogis(
    -0.5 + 0.6 * (age < 45) +
      0.4 * (education %in% c("Tertiary","Postgraduate")) +
      0.3 * (region %in% c("Greater Accra","Ashanti"))
  ))

  has_overdraft_facility <- rbinom(n, 1, prob = plogis(
    -2.5 + 0.7 * (account_type %in% c("Premium","Private Banking")) +
      0.4 * (employment_status == "Business Owner")
  ))

  # -- SECTION 4: BALANCES AND TRANSACTIONS ------------------------------
  savings_balance <- round(
    pmax(0, monthly_income) *
      runif(n, 0.5, 8) *
      case_when(
        account_type == "Basic"           ~ 0.4,
        account_type == "Standard"        ~ 1.0,
        account_type == "Premium"         ~ 2.5,
        account_type == "Private Banking" ~ 6.0,
        TRUE ~ 1.0
      ) *
      rlnorm(n, 0, 0.6),
    2
  )

  current_balance <- ifelse(
    has_current_account == 1,
    round(monthly_income * runif(n, 0.2, 2.0) * rlnorm(n, 0, 0.5), 2),
    0
  )

  avg_monthly_transactions <- rpois(n, lambda = pmax(1,
                                                     3 + 0.002 * monthly_income +
                                                       5 * has_mobile_banking +
                                                       3 * (account_type %in% c("Premium
                                                           ","Private Banking"))
  ))

  avg_transaction_value <- round(
    pmax(5, monthly_income / pmax(1, avg_monthly_transactions) *
           rlnorm(n, 0, 0.4)),
    2
  )

  atm_withdrawals_monthly <- rpois(n, lambda = pmax(0.2,
                                                    4 - 1.5 * has_mobile_banking + 0.5 *
                                                         (age > 50)
  ))

  online_transactions_monthly <- rpois(n, lambda = pmax(0,
                                                        2 + 8 * has_mobile_banking +
                                                          0.001 * monthly_income -
                                                          0.05 * age
  ))

  international_transactions_yearly <- rpois(n, lambda = pmax(0,
                                                              0.2 + 0.0008 * monthly_
                                                                  income +
                                                                1.5 * (account_type %in%
```

```
                                                                    c("Premium","
                                                                   Private Banking"))
))

overdraft_usage_yearly <- ifelse(
  has_overdraft_facility == 1,
  rpois(n, lambda = pmax(0, 2 + 0.3 * (employment_status=="Business Owner"))),
  0
)

# -- SECTION 5: CREDIT & RISK PROFILE -----------------------------------
credit_score_base <- 650 +
  1.5 * (log1p(monthly_income) - 7) * 20 +
  0.3 * age -
  30 * has_overdraft_facility -
  15 * overdraft_usage_yearly +
  rnorm(n, 0, 40)

credit_score <- round(pmax(300, pmin(850, credit_score_base)))

num_active_loans <- has_personal_loan + has_mortgage +
  rbinom(n, 1, prob = plogis(-2 + 0.3*(employment_status=="Business Owner")))

total_outstanding_debt <- round(
  pmax(0,
       has_personal_loan * runif(n, 500, 15000) +
         has_mortgage      * runif(n, 20000, 250000) +
         has_overdraft_facility * overdraft_usage_yearly * runif(n, 100, 2000)
  ),
  2
)

debt_to_income_ratio <- round(
  total_outstanding_debt / pmax(1, monthly_income * 12),
  3
)

num_missed_payments <- rpois(n, lambda = pmax(0,
                                              0.3 + 0.02 * debt_to_income_ratio * 10 -
                                                0.005 * (credit_score - 600)
))

default_prob <- plogis(
  -3.0 +
    0.04 * num_missed_payments +
    1.5  * (debt_to_income_ratio > 0.5) -
    0.01 * (credit_score - 600) +
    0.3  * (employment_status %in% c("Unemployed","Student"))
)
loan_default_flag <- rbinom(n, 1, prob = pmin(default_prob, 0.6))

# -- SECTION 6: CUSTOMER BEHAVIOUR & ENGAGEMENT -------------------------
tenure_years <- round(as.numeric(as.Date("2024-12-31") - account_open_date) / 365.25,
    2)

satisfaction_score <- pmin(10, pmax(1, round(
  7.0 -
    0.5 * num_missed_payments +
    0.3 * has_mobile_banking +
    0.2 * (account_type %in% c("Premium","Private Banking")) +
    rnorm(n, 0, 1.5)
)))

branch_visits_yearly <- rpois(n, lambda = pmax(0,
                                               6 - 4 * has_mobile_banking + 0.05 * age
))

num_complaints <- rpois(n, lambda = pmax(0,
                                         0.2 + 0.3 * num_missed_payments +
                                           0.4 * (satisfaction_score < 4)
))

churn_prob <- plogis(
  -2.0 -
    0.3 * (satisfaction_score - 5) +
```

```
      0.2 * num_complaints -
      0.3 * has_mobile_banking -
      0.05 * tenure_years +
      0.2 * (account_type == "Basic")
  )
  churned <- rbinom(n, 1, prob = pmin(churn_prob, 0.7))

  # -- SECTION 7: FREE-TEXT FEEDBACK ---------------------------------------
  feedback_templates <- c(
    "The mobile app keeps crashing whenever I try to make a transfer.",
    "My loan application took far too long to be approved.",
    "Customer service at the branch was excellent and very helpful.",
    "I was charged a fee I did not understand on my statement.",
    "The new online banking interface is confusing to navigate.",
    "Staff at the branch were rude and unprofessional during my visit.",
    "I appreciate the quick response when I reported a lost card.",
    "Interest rates on savings accounts are too low compared to competitors.",
    "The ATM near my home is frequently out of cash.",
    "Setting up the fixed deposit was smooth and well explained.",
    "I have been waiting for a refund on a failed transaction for weeks.",
    "The credit card application process was straightforward and fast.",
    "There is no clear information about overdraft charges.",
    "I love the new budgeting feature in the mobile app.",
    "My relationship manager rarely responds to my emails."
  )

  customer_feedback <- sample(feedback_templates, n, replace = TRUE)
  customer_feedback[sample(1:n, size = round(n * 0.15))] <- NA_character_

  # -- SECTION 8: INTENTIONAL DATA QUALITY ISSUES --------------------------
  monthly_income[sample(1:n, size = round(n * 0.06))] <- NA_real_
  credit_score[sample(1:n, size = round(n * 0.04))] <- NA_real_
  dependents[sample(1:n, size = round(n * 0.03))] <- NA_integer_
  satisfaction_score[sample(1:n, size = round(n * 0.05))] <- NA_real_

  age[sample(1:n, size = round(n * 0.001))] <- sample(c(1, 5, 130, 150), round(n*0.001),
      replace = TRUE)
  monthly_income[sample(1:n, size = round(n * 0.001))] <- sample(c(0, -500, 5000000),
      round(n*0.001), replace = TRUE)
  savings_balance[sample(1:n, size = round(n * 0.0015))] <-
    savings_balance[sample(1:n, size = round(n * 0.0015))] * 1000
  debt_to_income_ratio[sample(1:n, size = round(n * 0.0008))] <- -abs(
    debt_to_income_ratio[sample(1:n, size = round(n * 0.0008))]
  )

  gender_messy <- gender
  messy_idx <- sample(1:n, size = round(n * 0.02))
  gender_messy[messy_idx] <- case_when(
    gender_messy[messy_idx] == "Male"   ~ sample(c("male","MALE","M"), length(messy_idx),
        replace=TRUE),
    gender_messy[messy_idx] == "Female" ~ sample(c("female","FEMALE","F"), length(messy_
        idx), replace=TRUE),
    TRUE ~ gender_messy[messy_idx]
  )

  region_messy <- region
  messy_idx2 <- sample(1:n, size = round(n * 0.015))
  region_messy[messy_idx2] <- trimws(paste0(" ", region_messy[messy_idx2], "  "))

  bad_date_idx <- sample(1:n, size = round(n * 0.0005))
  account_open_date[bad_date_idx] <- as.Date("2026-01-01") + days(sample(1:300, length(
      bad_date_idx), replace=TRUE))

  # -- ASSEMBLE FINAL DATA FRAME -------------------------------------------
  df <- data.frame(
    customer_id, branch_id, relationship_manager_id, account_open_date,
    age, gender = gender_messy, region = region_messy, education,
    employment_status, marital_status, dependents, monthly_income,
    account_type, has_savings_account, has_current_account, has_fixed_deposit,
    has_credit_card, has_personal_loan, has_mortgage, has_investment_account,
    has_insurance_product, has_mobile_banking, has_overdraft_facility,
    savings_balance, current_balance, avg_monthly_transactions,
    avg_transaction_value, atm_withdrawals_monthly, online_transactions_monthly,
    international_transactions_yearly, overdraft_usage_yearly,
    credit_score, num_active_loans, total_outstanding_debt,
```

```r
    debt_to_income_ratio, num_missed_payments, loan_default_flag,
    tenure_years, satisfaction_score, branch_visits_yearly, num_complaints,
    churned, customer_feedback,
    stringsAsFactors = FALSE
  )

  dup_idx <- sample(1:nrow(df), size = round(nrow(df) * 0.003))
  df <- bind_rows(df, df[dup_idx, ])

  return(df)
}

df_raw <- simulate_bdat602_project(n = 100000, seed = 602)
cat("Generated dataset:", nrow(df_raw), "rows x", ncol(df_raw), "columns\n")

#'
#' # Variable Selection
#'
#' **(a) Segmentation problem.** This task investigates how retail banking customers can be
#' grouped into distinct **risk profiles** -- e.g. low-risk, financially stable customers versus
#' high-risk, over-leveraged or delinquency-prone customers -- so the bank can differentiate credit
#' exposure management, pricing, and collections strategies across segments.
#'
#' **(b) Selected variables.** The cluster analysis draws on `monthly_income`, `savings_balance`,
#' `current_balance`, `credit_score`, `num_active_loans`, `total_outstanding_debt`,
#' `debt_to_income_ratio`, `num_missed_payments`, and `overdraft_usage_yearly`. Jointly these
#' capture three complementary dimensions of risk: (i) **repayment capacity** (income, savings and
#' current balances as buffers against shocks), (ii) **leverage** (outstanding debt, DTI, number of
#' active loans), and (iii) **repayment behaviour** (missed payments, overdraft usage, credit score
#' as a composite behavioural summary).
#'
#' **(c) Excluded alternatives.** Demographic fields (age, gender, education, region, marital
#' status) were excluded as they are not direct financial-risk indicators and raise
#' proxy-discrimination concerns. Identifiers (`customer_id`, `branch_id`,
#' `relationship_manager_id`) were dropped as meaningless for clustering. Engagement/service
#' variables (`satisfaction_score`, `branch_visits_yearly`, `num_complaints`, `churned`,
#' `has_mobile_banking`) were excluded as they reflect service experience rather than credit risk.
#' `loan_default_flag` was excluded because it is the eventual supervised target and its inclusion
#' would leak the outcome into an unsupervised risk-clustering exercise.
#'
#' **(d) Correlation-based exclusions.** `has_personal_loan` and `has_mortgage` were dropped
#' because they are already summarised in `num_active_loans`. `avg_transaction_value` and
#' `avg_monthly_transactions` were excluded as they correlate strongly with `monthly_income` and
#' mainly reflect channel activity rather than risk.
#'
## ----select-vars-----------------------------------------------------------
clust_vars <- c("customer_id","monthly_income","savings_balance","current_balance",
                "credit_score","num_active_loans","total_outstanding_debt",
                "debt_to_income_ratio","num_missed_payments","overdraft_usage_yearly")

df <- df_raw

#'
#' # Data Cleaning
#'
#' ## Inspect data quality issues
#'
## ----inspect-issues--------------------------------------------------------
cat("=== Missing values ===\n")
print(sapply(df[clust_vars], function(x) sum(is.na(x))))

```

```
cat("\n=== Duplicate rows ===\n")
cat("Total rows:", nrow(df), " Unique customer_id:", length(unique(df$customer_id)), "\n")
cat("Fully duplicated rows:", sum(duplicated(df)), "\n")

cat("\n=== Implausible values ===\n")
cat("monthly_income <= 0 or > 30,000:",
    sum(df$monthly_income <= 0 | df$monthly_income > 30000, na.rm = TRUE), "\n")
cat("debt_to_income_ratio < 0:", sum(df$debt_to_income_ratio < 0, na.rm = TRUE), "\n")
cat("savings_balance > 99.5th percentile cap candidates:",
    sum(df$savings_balance > quantile(df$savings_balance, 0.995, na.rm = TRUE)), "\n")

#'
#' ## Issue 1 -- Exact duplicate records
#'
#' 300 fully duplicated rows (0.3%) are present, from a data-integration duplication step.
#' **Fix:** removed via `distinct()`. Deduplication is unambiguous here since the rows are
#' byte-for-byte identical, so removal loses no information and introduces no bias.
#'
## ----clean-duplicates-------------------------------------------------------
n_dup <- sum(duplicated(df))
df <- df %>% distinct()
cat(sprintf("Removed %d exact duplicate rows -> %d rows remain\n", n_dup, nrow(df)))

#'
#' ## Issue 2 -- Implausible / impossible values (data entry errors)
#'
#' - `monthly_income`: non-positive values and an implausible extreme spike (up to 5,000,000).
#'   Recoded to `NA` (handled with the genuine missing values in Issue 3).
#' - `debt_to_income_ratio`: negative ratios are mathematically impossible -- corrected via
#'   `abs()`, which exactly reverses the sign-flip entry error.
#' - `savings_balance`: "extra zero" entry errors (~1000x inflation) -- winsorised (capped) at the
#'   99.5th percentile rather than deleted, preserving the record while preventing a handful of
#'   rows from dominating distance-based clustering.
#'
## ----clean-implausible------------------------------------------------------
n_income_bad <- sum(df$monthly_income <= 0 | df$monthly_income > 30000, na.rm = TRUE)
df$monthly_income[df$monthly_income <= 0] <- NA
df$monthly_income[df$monthly_income > 30000] <- NA
cat(sprintf("Recoded %d implausible monthly_income values to NA\n", n_income_bad))

n_dti_neg <- sum(df$debt_to_income_ratio < 0, na.rm = TRUE)
df$debt_to_income_ratio <- abs(df$debt_to_income_ratio)
cat(sprintf("Corrected %d negative debt_to_income_ratio values via abs()\n", n_dti_neg))

sb_cap <- as.numeric(quantile(df$savings_balance, 0.995, na.rm = TRUE))
n_sb_out <- sum(df$savings_balance > sb_cap, na.rm = TRUE)
df$savings_balance <- pmin(df$savings_balance, sb_cap)
cat(sprintf("Winsorised %d extreme savings_balance values to the 99.5%% cap (%.0f)\n",
            n_sb_out, sb_cap))

#'
#' ## Issue 3 -- Missing values
#'
#' `monthly_income` (~6%) and `credit_score` (~4%) contain missing values. **Fix:** median
#' imputation, with a `*_was_missing` flag retained for each. Listwise deletion would discard
#' ~10% of customers and could bias segments if missingness relates to customer type; mean
#' imputation was rejected because both variables are heavily right-skewed, so the median better
#' represents a "typical" customer. The missingness flags preserve the information that a value
#' was imputed.
#'
## ----clean-missing----------------------------------------------------------
n_miss_income <- sum(is.na(df$monthly_income))
n_miss_credit <- sum(is.na(df$credit_score))
```

```
476
477 df$income_was_missing <- as.integer(is.na(df$monthly_income))
478 df$credit_was_missing <- as.integer(is.na(df$credit_score))
479
480 df$monthly_income[is.na(df$monthly_income)] <- median(df$monthly_income, na.rm = TRUE)
481 df$credit_score[is.na(df$credit_score)]     <- median(df$credit_score, na.rm = TRUE)
482
483 cat(sprintf("Median-imputed %d missing monthly_income and %d missing credit_score values\
    n",
484             n_miss_income, n_miss_credit))
485
486 cat("\nRemaining NAs in clustering variables:\n")
487 print(sapply(df[clust_vars], function(x) sum(is.na(x))))
488
489 #'
490 #' # Preprocessing
491 #'
492 #' Skewness is assessed empirically to choose an appropriate transform/scaling method per
      variable.
493 #'
494 ## ----skew-function-------------------------------------------------------------
495 skew <- function(x){ m <- mean(x); s <- sd(x); mean((x - m)^3) / s^3 }
496
497 #'
498 #' ## Group A -- highly right-skewed monetary variables: log1p + Z-score
499 #'
500 ## ----preprocess-log------------------------------------------------------------
501 log_vars <- c("monthly_income","savings_balance","current_balance",
502               "total_outstanding_debt","debt_to_income_ratio")
503
504 cat("Skewness BEFORE log transform:\n")
505 for (v in log_vars) cat(sprintf("  %-25s %.2f\n", v, skew(df[[v]])))
506
507 for (v in log_vars) df[[paste0(v, "_log")]] <- log1p(df[[v]])
508
509 cat("\nSkewness AFTER log1p transform:\n")
510 for (v in log_vars) cat(sprintf("  %-25s %.2f\n", v, skew(df[[paste0(v,"_log")]])))
511
512 z_scale <- function(x) as.numeric(scale(x))
513
514 #'
515 #' All five monetary variables were strongly right-skewed (skew up to 25.6), which would
      let a
516 #' handful of wealthy customers dominate Euclidean-distance-based clustering. Log-
      transforming
517 #' compresses the tail, and Z-scoring afterward puts all five on a comparable variance
      scale.
518 #' (‘debt_to_income_ratio‘ and ‘total_outstanding_debt‘ retain moderate residual skew
      even after
519 #' logging -- a structural zero-inflation issue, since many customers hold no debt at all
      , not
520 #' something a transform alone fixes.)
521 #'
522 #' ## Group B -- credit_score: robust scaling (median/IQR)
523 #'
524 #' ‘credit_score‘ has a healthy, roughly symmetric spread with a non-zero IQR, so median/
      IQR
525 #' scaling works as intended and down-weights the small share of extreme low scores
      without being
526 #' distorted by them.
527 #'
528 ## ----preprocess-robust---------------------------------------------------------
529 robust_scale <- function(x) {
530   med <- median(x); iqr <- IQR(x)
531   (x - med) / iqr
532 }
533 cat("credit_score IQR:", IQR(df$credit_score), "\n")
534
535 #'
536 #' ## Group C -- zero-inflated count variables: Min-Max normalisation
537 #'
538 #' ‘num_active_loans‘, ‘num_missed_payments‘, and ‘overdraft_usage_yearly‘ are heavily
539 #' zero-inflated: their median AND IQR are both 0 (>=75% of customers have zero missed
540 #' payments/overdraft use), so median/IQR robust scaling is undefined in practice. Min-
      Max avoids
```

```
#' relying on a spread statistic and simply bounds each variable to [0,1].
#'
## ----preprocess-minmax-------------------------------------------------------
minmax_vars <- c("num_active_loans","num_missed_payments","overdraft_usage_yearly")
minmax_scale <- function(x) (x - min(x)) / (max(x) - min(x))

for (v in c("num_active_loans","num_missed_payments","overdraft_usage_yearly")) {
  cat(v, " IQR:", IQR(df[[v]]), " MAD:", mad(df[[v]]), "\n")
}

#'
#' ## Build final feature matrix
#'
## ----build-features----------------------------------------------------------
features <- data.frame(customer_id = df$customer_id)
for (v in log_vars) features[[paste0(v, "_z")]] <- z_scale(df[[paste0(v, "_log")]])
features[["credit_score_robust"]] <- robust_scale(df[["credit_score"]])
for (v in minmax_vars) features[[paste0(v, "_minmax")]] <- minmax_scale(df[[v]])

cat("Final feature matrix:", nrow(features), "rows x", ncol(features) - 1, "features\n")
str(features)

#'
#' # Cluster Analysis
#'
## ----cluster-setup-----------------------------------------------------------
suppressMessages({
  library(cluster)
  library(dbscan)
  library(ggplot2)
})
X <- as.matrix(features[, -1])
set.seed(602)

#'
#' ## K-means -- (a) Elbow method
#'
## ----kmeans-elbow, cache=TRUE------------------------------------------------
k_range <- 1:10
set.seed(602)
wss <- sapply(k_range, function(k) {
  km <- kmeans(X, centers = k, nstart = 10, iter.max = 50)
  km$tot.withinss
})
elbow_df <- data.frame(k = k_range, wss = wss)
elbow_df$pct_drop <- c(NA, round(100 * diff(-elbow_df$wss) / head(-elbow_df$wss, -1), 1))
print(elbow_df)

#'
## ----elbow-plot--------------------------------------------------------------
ggplot(elbow_df, aes(k, wss)) +
  geom_line(color = "#2C6E9E", linewidth = 1) +
  geom_point(size = 2, color = "#2C6E9E") +
  geom_vline(xintercept = 6, linetype = "dashed", color = "grey40") +
  scale_x_continuous(breaks = 1:10) +
  labs(title = "K-means Elbow Method", x = "Number of clusters (k)",
       y = "Total within-cluster sum of squares (WSS)") +
  theme_minimal(base_size = 13)

#'
#' The steep declines through k = 2-5 flatten out sharply from **k = 6 onward** -- the elbow sits
#' at k = 5 or 6.
#'
#' ## K-means -- (b) Silhouette method
#'
#' Computed on a random 8,000-customer sample (silhouette is O(n^2), infeasible on the full
#' 100,000 rows); cluster assignments themselves come from k-means fit on the **full** dataset.
#'
## ----kmeans-silhouette, cache=TRUE-------------------------------------------
set.seed(602)
samp_idx <- sample(nrow(X), 8000)
Xs <- X[samp_idx, ]
```

```
613
614 sil_width <- sapply(2:10, function(k) {
615   km <- kmeans(X, centers = k, nstart = 10, iter.max = 50)
616   km_labels_sample <- km$cluster[samp_idx]
617   ss <- silhouette(km_labels_sample, dist(Xs))
618   mean(ss[, 3])
619 })
620 sil_df <- data.frame(k = 2:10, avg_silhouette = round(sil_width, 4))
621 print(sil_df)
622
623 #'
624 ## ----silhouette-plot-----------------------------------------------------
625 ggplot(sil_df, aes(k, avg_silhouette)) +
626   geom_line(color = "#C24D2C", linewidth = 1) +
627   geom_point(size = 2, color = "#C24D2C") +
628   geom_vline(xintercept = 6, linetype = "dashed", color = "grey40") +
629   scale_x_continuous(breaks = 2:10) +
630   labs(title = "K-means Silhouette Method", x = "Number of clusters (k)",
631        y = "Average silhouette width (sample n=8,000)") +
632   theme_minimal(base_size = 13)
633
634 #'
635 #' The global maximum is at k = 2 (0.293), but there is a clear local peak at **k = 6-7**
636 #' (0.286-0.288), after a dip at k = 3.
637 #'
638 #' ## (c) Reconciling elbow vs. silhouette
639 #'
640 #' k = 2 is statistically the tightest split but merely separates "low debt" from "high
      debt /
641 #' elevated DTI" customers -- too coarse for risk-based segmentation. **k = 6** sits at a
      local
642 #' silhouette peak nearly as high as the global maximum, and is exactly where the elbow's
      marginal
643 #' WSS gains flatten out. Profiling confirms k = 6 is business-meaningful, isolating a
      distinct
644 #' high-risk cluster. **Final choice: k = 6.**
645 #'
646 ## ----kmeans-final--------------------------------------------------------
647 set.seed(602)
648 km6 <- kmeans(X, centers = 6, nstart = 10, iter.max = 50)
649 cat("Cluster sizes:\n")
650 print(table(km6$cluster))
651 cat("\nCluster centers (scaled features):\n")
652 print(round(km6$centers, 2))
653
654 #'
655 #' ## Second clustering method: DBSCAN
656 #'
657 #' **Why it complements K-means:** K-means assumes spherical, similarly-sized clusters
      and forces
658 #' every customer into one of k centroids -- genuine outliers get averaged into the
      nearest group,
659 #' diluting exactly the extreme-risk customers a risk-classification exercise needs to
      isolate.
660 #' DBSCAN makes no shape assumption and explicitly separates dense "normal" regions from
      sparse
661 #' outlier points, labelling the latter as **noise**.
662 #'
663 #' `eps` was chosen from the k-distance elbow (minPts = 2 x dimensionality = 18):
664 #'
665 ## ----dbscan-eps-selection, cache=TRUE------------------------------------
666 minPts <- 2 * ncol(X)
667 kdist <- sort(kNNdist(X, k = minPts))
668 qs <- seq(0.85, 0.99, by = 0.02)
669 print(round(quantile(kdist, qs), 3))
670
671 #'
672 ## ----dbscan-final, cache=TRUE--------------------------------------------
673 db <- dbscan(X, eps = 0.60, minPts = minPts)
674 cat("DBSCAN cluster sizes (0 = noise):\n")
675 print(table(db$cluster))
676
677 #'
678 #' **Advantage for this data:** comparing DBSCAN's noise points against clustered points
      on the
```

```
#' raw (unscaled) risk variables shows the noise set is dramatically worse on every indicator
#' simultaneously -- DBSCAN surfaced this structure automatically, without being told to look for
#' "risky" customers.
#'
## ----dbscan-profile-----------------------------------------------------------
noise_flag <- db$cluster == 0
compare_vars <- c("debt_to_income_ratio","total_outstanding_debt","num_missed_payments",
                  "overdraft_usage_yearly","credit_score","monthly_income")
for (v in compare_vars) {
  cat(sprintf("%-25s noise mean=%10.2f | clustered mean=%10.2f\n",
              v, mean(df[[v]][noise_flag]), mean(df[[v]][!noise_flag])))
}

#'
#' ## Visual comparison (PCA projection)
#'
## ----pca-plots, fig.width=8, fig.height=6-------------------------------------
set.seed(602)
pca <- prcomp(X, center = FALSE, scale. = FALSE)
var_exp <- round(100 * (pca$sdev^2 / sum(pca$sdev^2))[1:2], 1)

plot_df <- data.frame(PC1 = pca$x[,1], PC2 = pca$x[,2],
                      kmeans_cluster = factor(km6$cluster),
                      dbscan_cluster = factor(ifelse(db$cluster == 0, "Noise", db$cluster)))
plot_samp <- plot_df[sample(nrow(plot_df), 12000), ]

ggplot(plot_samp, aes(PC1, PC2, color = kmeans_cluster)) +
  geom_point(alpha = 0.4, size = 0.8) +
  labs(title = "K-means (k=6) clusters -- PCA projection",
       x = paste0("PC1 (", var_exp[1], "%)"), y = paste0("PC2 (", var_exp[2], "%)"),
       color = "Cluster") +
  theme_minimal(base_size = 13)

ggplot(plot_samp, aes(PC1, PC2, color = dbscan_cluster)) +
  geom_point(alpha = 0.4, size = 0.8) +
  scale_color_manual(values = c("1"="#2C6E9E","2"="#C24D2C","3"="#3A9E4D",
                                "4"="#8E5EA2","5"="#D4A017","Noise"="grey70")) +
  labs(title = "DBSCAN clusters -- PCA projection (grey = noise/outliers)",
       x = paste0("PC1 (", var_exp[1], "%)"), y = paste0("PC2 (", var_exp[2], "%)"),
       color = "Cluster") +
  theme_minimal(base_size = 13)

#'
#' ## Save final cluster assignments
#'
## ----save-outputs-------------------------------------------------------------
assignments <- data.frame(
  customer_id = features$customer_id,
  kmeans_k6_cluster = km6$cluster,
  dbscan_cluster = db$cluster
)
write.csv(assignments, "final_cluster_assignments.csv", row.names = FALSE)
cat("Saved final_cluster_assignments.csv (", nrow(assignments), "rows)\n")

#'
#' # Summary
#'
#' - **Data generation** produced ~100,300 simulated customer records with realistic banking
#'   relationships and embedded data-quality issues.
#' - **Variable selection** focused on nine variables spanning repayment capacity, leverage, and
#'   repayment behaviour, chosen to serve a risk-segmentation business objective.
#' - **Cleaning** removed 300 exact duplicates, corrected implausible/impossible values in three
#'   variables, and median-imputed missing income and credit-score values (~10% combined).
#' - **Preprocessing** applied log1p + Z-score to skewed monetary variables, robust (median/IQR)
#'   scaling to credit score, and Min-Max scaling to zero-inflated count variables, based on
#'   empirically checked skewness and spread statistics.
```

```
#' - **K-means (k = 6)**, chosen by reconciling the elbow and silhouette diagnostics against
#'   business interpretability, produced six segments including a distinct high-risk group
#'   (~4% of customers, elevated DTI and missed payments).
#' - **DBSCAN** complemented K-means by isolating ~3.3% of customers as density-based outliers,
#'   who show sharply worse values on every raw risk indicator -- a segment K-means' centroid-based
#'   approach would otherwise average away.
```